Subject Code	Subject Name	Lectures (Periods)	Tutorials (Periods)	Practical (Periods)
CS P62	EMBEDDED SYSTEMS LABORATORY	-	-	3

LIST OF EXPERIMENTS

The following programs are to be implemented on ARM based Processors/Equivalent.

1. Simple Assembly Program for Addition, Subtraction, Multiplication and Division
2. Simple Assembly Program for System Calls and Interrupts, Loops and Branches
3. Write an Assembly programs to configure and control General Purpose Input/Output (GPIO) port pins.
4. Write an Assembly programs to read digital values from external peripherals and execute them with the Target board.
5. Program to demonstrate Time delay program using built in Timer / Counter feature on IDE environment
6. Program to demonstrate a simple interrupt handler and setting up a timer
7. Program to Interface 8 Bit LED and Switch Interface
8. Program to implement Buzzer Interface on IDE environment Program to Displaying a message in a 2 line x 16 Characters LCD display and verify the result in debug terminal.
9. Program to demonstrate I^2C Interface on IDE environment
10. Program to demonstrate I^2C Interface – Serial EEPROM
11. Demonstration of Serial communication. Transmission from Kit and reception from PC using
12. Serial Port on IDE environment use debug terminal to trace the program.

Write the following programs to understand the use of RTOS with ARM Processor on IDE Environment using ARM Tool chain and Library:

1. Write an application that creates a task which is scheduled when a button is pressed, which illustrates the use of an event set between an ISR and a task
2. Write an application that Demonstrates the interruptible ISRs(Requires timer to have higher priority than external interrupt button)
3. Write an application that creates a two task to Blinking two different LEDs at different timings
4. Write an application that creates a two task displaying two different messages in LCD display in two lines.
5. Sending messages to mailbox by one task and reading the message from mailbox by another task.
6. Sending message to PC through serial port by three different tasks on priority Basis.
7. Basic Audio Processing on IDE environment.

ASM (add, Sub, Mul, Div (or) Arithmetic)

Step 1: Open Keil → Project → close project

Step 2: Project → New µvision project → Create folder → Save the pgm (eg: add)
☑ SSS (Inside of folder)

Step 3: Save pgm → NXP (founded by Philips) → LPC2148 → OK → yes

Step 4: File → new → type pgm → Save (as add.asm)

Step 5: (Project Workspace) Source group Right Click → add files to group Source group1 → Click asm file (or) add.asm → Add → Close

Step 6: Built target → Debug → Run → Halt
File name: add.asm
File type: Asm source file

Step 7: Output display

ARM:-

Same Step 1 to Step 6 (Built) follow in ARM pgm

Step 7: flash → Configure flash tools → output → Tick ☑ Create HEX file

Step 8: Open flash magic → Comport : Com 1
Baudrate : 9600
Device - LPC2148
Interface : None (ISP)
Oscilator : 12
☑ erase all flash + Code Rd Prot

Step 9: Browse → Select that HEX file → OK

Step 10: Kit → Press power button (white) → two times Press the Restart button → Flash magic Start

Step 11: Once Program finished → kit → power off → Press 1 time Reset → on / power

1. *Introduction*

Thank you for purchasing the **PRIMER-ARM214x** Kit. You will find it useful in developing your ARM7 application.

PRIMER-ARM214X Kit, is proposed to smooth the progress of developing and debugging of various designs encompassing of High speed 32-bit MCU from NXP. The board supports NXP's LPC214x family devices with various memory and peripheral options. It integrates on board two UARTs, LEDs, Relays, Motor Interface, keypads, an ADC input and GLCD/LCD Display to create a stand-alone versatile test platform.

1.1– Packages

- PRIMER-ARM214X Kit (LPC2148 MCU)
- Serial Port Cable
- Printed User Manual
- CD contains
- Software (Programmers, IDE)
  - Example Programs
    User Manual

Buzzer -> put chip last two pin

LED -> on the buth light blink

LCD -> on LCD but in red sy

I2C7 seg ->

---

2. *Specifications*

MCU	NXP's ARM7TDMI LPC2148 MCU
Memory	512K Flash – Program Memory 32K+8K RAM – Data Memory
Clock	12MHz crystal for maximum (5xPLL = 60MHz CPU clock) | 32 KHz RTC crystal
On-Board Peripherals	• 8 Nos. Point LEDs • 8 Nos. Digital Input(Slide Switch) • 4x4 Matrix Keypad • 2X16 Character LCD with back Light

- 4 Nos. 7-Segment Display (I2C)
- 2 Nos. Analog Input (Potentiometer)
- Temperature Sensor
- Stepper Motor Interface
- 2 Nos. of SPDT Relay
- RTC with Batter-Backup
- 2 Nos. UART(RS232)
- USB 2.0 device interface
- Buzzer (Alarm)
- PS/2 (keyboard interface)
- Digital/Analog Output
- Interrupts Study, Reset Button

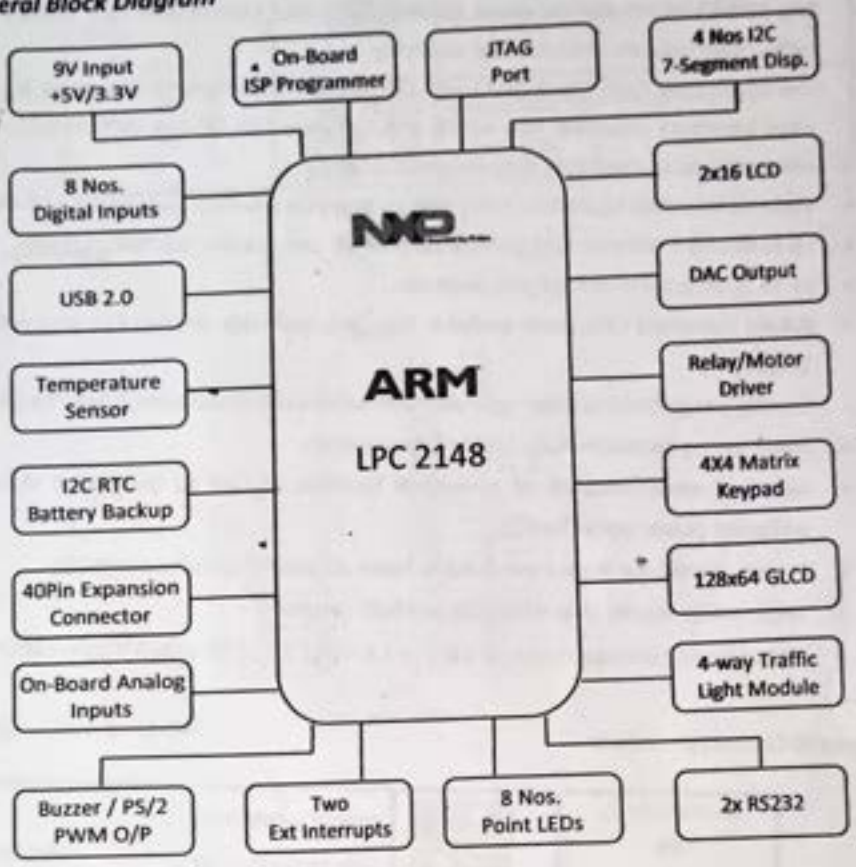
Power

- 9-12V, AC/DC- Adaptors,
  Power form USB (+5V) (+3.3V, 800mA)

Connectors

- JTAG (Programming/ Debugging)
- D-SUB Connector (Serial Port, ISP)
- 40 – PIN Expansion Connector
- Ext Analog Input Connector

2.1- General Block Diagram



2.2 - LPC2148 Processor Features

- 16-bit/32-bit ARM7TDMI-S microcontroller in a tiny LQFP64 package.
- 8 kB to 40 kB of on-chip static RAM and 32 kB to 512 kB of on-chip flash memory. 128-bit wide interface/accelerator enables high-speed 60 MHz operation.
- In-System Programming/In-Application Programming (ISP/IAP) via on-chip boot loader software. Single flash sector/full chip erase in 400 ms and programming of 256 bytes in 1 ms.
- USB 2.0 Full-speed compliant device controller with 2 kB of endpoint RAM. In addition, the LPC2146/48 provides 8 kB of on-chip RAM accessible to USB by DMA.
- One or two (LPC2141/42 vs. LPC2144/46/48) 10-bit ADCs provide a total of 6/14 analog inputs, with conversion times as low as 2.44 µs per channel.

- Single 10-bit DAC provides variable analog output (LPC2142/44/46/48 only).
- Two 32-bit timers/external event counters (with four capture and four compare channels each), PWM unit (six outputs) and watchdog.
- Low power Real-Time Clock (RTC) with independent power and 32 kHz clock input. Multiple serial interfaces including two UARTs (16C550), two Fast I2C-bus (400 kbit/s), SPI and SSP with buffering and variable data length capabilities.
- Vectored Interrupt Controller (VIC) with configurable priorities and vector addresses.
- Up to 45 of 5 V tolerant fast general purpose I/O pins in a tiny LQFP64 package.
- Up to 21 external interrupt pins available.
- 60MHz maximum CPU clock available from programmable on-chip PLL with settling time of 100µs.
- On-chip integrated oscillator operates with an external crystal from 1 MHz to 25 MHz.
- Power saving modes include Idle and Power-down.
- Individual enable/disable of peripheral functions as well as peripheral clock scaling for additional power optimization.
- Processor wake-up from Power-down mode via external interrupt or BOD.
- Single power supply chip with POR and BOD circuits:
- CPU operating voltage range of 3.0 V to 3.6 V (3.3 V ± 10 %) with 5 V tolerant I/O pads.

3. Jumper & Connector Details

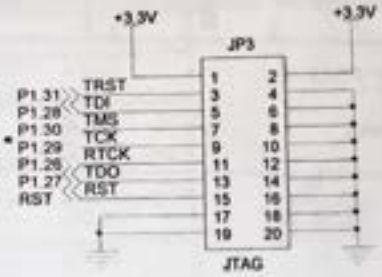
Stepper / Relay **JP8**		Internal Supply (+5V) External Supply(+5V)
Analog I/P (P0.29) **JP4**		On-Board Analog Input(+3.3V) External Analog Input-1 select
Analog I/P (P0.30) **JP5**		On-Board Analog Input(+3.3V) External Analog Input-2 select
Buzzer (P0.7) **JP1**		Enable Buzzer Disable Buzzer
JTAG **JP6**		Enable JTAG Disable Power JTAG

USB Voltage Read JP13		Enable/Disable USB Voltage Read
LED J4		Enable LEDs, Port (P1.16 – P1.23) Disable LEDs.

Connector Details

40-Pin Expansion Connector

JTAG Connector



4. Power Supply

The external power can be AC or DC, with a voltage between (9V/12V,1A output) at 230V AC input. The ARM board produces +5V using an LM7805 voltage regulator, which provides supply to the peripherals. LM1117 Fixed +3.3V positive regulator used for processor & processor related peripherals. USB socket meant for power supply and USB communication, user can select either USB or Ext power supply through JP14. Separate On/Off Switch (SW24) for controlling power to the board.

+5V USB/EXT SW1		Power +5V (EXT through Adaptor) Power +5V (USB)

5. Flash Programming Utility

1. NXP (Philips)

NXP Semiconductors produce a range of Microcontrollers that feature both on-chip Flash memory and the ability to be reprogrammed using In-System Programming technology.

Program/Execution Mode

ISP Programming **J11**		Program Mode (LED on) Execution Mode
UART-0 / ISP PGM **P1 (DB-9 Male)**		

6. On-board Peripherals

The Development kit comes with many interfacing options

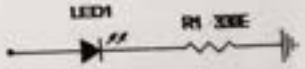
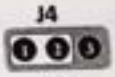
- 8-Nos. of Point LED's (Digital Outputs)

- 8-Nos. of Digital Inputs (slide switch)
- 2 Lines X 16 Character LCD Display
- I2C Enabled 4 Digit Seven-segment display
- 128x64 Graphical LCD Display
- 4 X 4 Matrix keypad
- Stepper Motor Interface
- 2 Nos. Relay Interface
- Two UART for serial port communication through PC
- Serial EEPROM
- On-chip Real Time Clock with battery backup
- PS/2 Keyboard interface(Optional)
- Temperature Sensor
- Buzzer(Alarm Interface)
- Traffic Light Module(Optional)

6.1 - *Light Emitting Diodes*

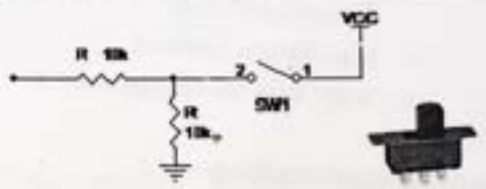
- Light Emitting Diodes (LEDs) are the most commonly used components, usually for displaying pin's digital states.

- The **ARM214X Kit** has 8 nos., of Point LEDs, connected with port pins (P1.16 to P1.23), to make port pins high LED will glow.

	Point LEDs	LPC2148 Lines	LED Selection
DIGITAL OUTPUTS	LD1	P1.16	 J4  - LED Enable J4  - LED Disable
	LD2	P1.17	
	LD3	P1.18	
	LD4	P1.19	
	LD5	P1.20	
	LD6	P1.21	
	LD7	P1.22	
	LD8	P1.23	

6.2 – Digital Inputs

- This is another simple interface, of 8-Nos. of slide switch, mainly used to give an input to the port lines, and for some control applications also.
- The **ARM214X Kit**, slide switches (SW20 to SW27) is connected with port pins (P1.24 to P1.31), user can give logical inputs **'LOW'**.
- The switches are connected to +3.3V, in order to detect a switch state, pull-down resistor should be used.

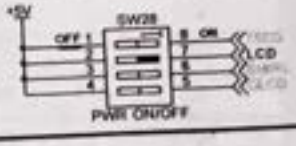
	Slide Switch	LPC2148 Lines	Input Logic Selection
DIGITAL INPUTS	SW20	P1.24	
	SW21	P1.25	
	SW22	P1.26	
	SW23	P1.27	
	SW24	P1.28	
	SW25	P1.29	
	SW26	P1.30	
	SW27	P1.31	

Make Switch Close – Low
Make Switch Open – High

6.3 - LCD 2x16 IN 4-BIT MODE

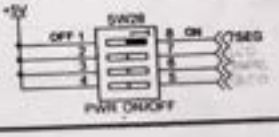
The **ARM214X Kit**, have 2x16 character LCD. 7 pins are needed to create 4-bit interface; 4 data bits (P0.19 – P0.22, D4-D7), address bit (RS-P0.16), read/write bit (R/W-P0.17) and control signal (E-P0.18). The LCD controller is a standard KS0070B or equivalent, which is a very well-known interface for smaller character based LCDs.

- Figure below illustrate the LCD part of the design and which pins are used for the interface. The LCD is powered from the 5V power supply enabled by switch **SW28**.

	LCD MODULE	LPC2148 LINES	2x16 LCD Selection
CONTROL	RS	P0.16	
CONTROL	RW	P0.17	
CONTROL	E	P0.18	
DATA LINES	D0-D3	NC	
DATA LINES	D4	P0.19	
DATA LINES	D5	P0.20	
DATA LINES	D6	P0.21	
DATA LINES	D7	P0.22	
 Make switch **SW28** to **'LCD'** label marking position			

6.4 –I2C Seven Segment Display

In **ARM214X Kit**, 4 nos. of common anode seven segment displays are controlled by I2C Enabled drivers. I2C Lines serial clock **SCL (P0.2)**, serial data **SDA (P0.3)** connected to the I2C based 7-segment display driver. The digit select lines are (MX1, MX2) controlled by the driver chip. The 7-segmend display is powered from the 5V power supply enabled by switch **SW28**.

	7-SEG Driver	LPC2148 LINES	7-SEG PWR Selection
7-SEG Display	SCL	P0.2	
7-SEG Display	SDA	P0.3	
 Make switch **SW28** to **'7SEG'** label marking position			

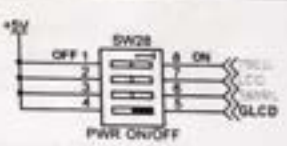
6.5 - 128x64 GLCD Graphical LCD

The **ARM214X Kit** is the GLCD. **14** pins are needed to create 8-bit interface; 8 data bits (P0.8 – P0.15, DB0-DB7), two chip select line P0.0(CS1) and P0.1(CS2), address bit (R/S-P0.4), read/write bit (R/W-P0.5) and control signal (E-P0.6) and Reset (RST-P0.7). The GLCD controller is a standard S6B0108 or equivalent, which is a very well-known interface for Graphical based LCDs.

Figure below illustrate the GLCD part of the design and which pins are used for the interface. The GLCD is powered from the 5V power supply enabled by switch **SW28**.

	GLCD	LPC2148 LINES	128x64 GLCD Selection
CONTROL LINES	CS1	P0.0	
	CS2	P0.1	
	RS	P0.4	
	R/W	P0.5	
	E	P0.6	
LCD – DATA LINES	DB0	P0.8	
	DB1	P0.9	
	DB2	P0.10	
	DB3	P0.11	
	DB4	P0.12	
	DB5	P0.13	
	DB6	P0.14	
	DB7	P0.15	
	RST	P0.7	

Make switch **SW28** and **SW30** to 'GLCD' label marking position

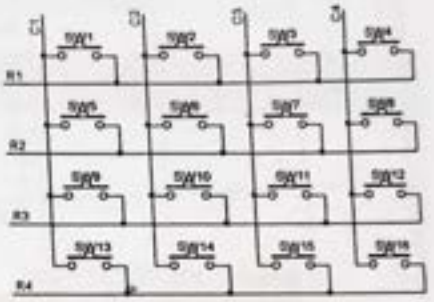


Pin Details of GLCD

Pin No.	Symbol	I/O Type	Description
1 2	/CSA /CSB	I	Chip selection When CS1=L,CS2=H, select IC1 When CS1=H,CS2=L, select IC2
3	VSS	Supply	Ground
4	VDD	Supply	Power supply
5	V0	Supply	LCD driver supply voltage
6	D/I		Data input/output pin of internal shift register MS SHL DIO1 DIO2 H H Output Output H L Output Output L H Input Output L L Output Input
7	R/W		Read or Write RW Description H Data appears at DB[7:0] and can be read by the CPU while E= H CS1B=L,CS2B=L and CS3=H. L Display data DB[7:0] can be written at falling edge of E when CS1B=L, CS2B=L and CS3=H.
8	E		Enable signal E Description H Read data in DB[7:0] appears while E= "High". L Display data DB[7:0] is latched at falling edge of E.
9~16	DB0~ DB7	I/O	Data bus [0~7] Bi-directional data bus
17	/RST	I	Reset signal. When RSTB=L [1] ON/OFF register becomes set by 0 (display off) [2] display start line register becomes set by 0 (Z-address 0 set, display from line 0) [3] After releasing reset , this condition can be changed only by instruction.
18	VEE	Power	VEE is connected by the same voltage.
19	A		Back-light anode
20	K		Back-light cathode

6.6 - 4x4 Matrix keypad

Keypads arranged by matrix format, each row and column section pulled by high or low by selection JS, all row lines(P1.24 – P1.27) and column lines(P1.28 to P1.31) connected directly by the port pins.

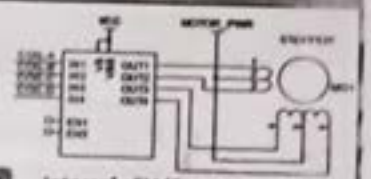
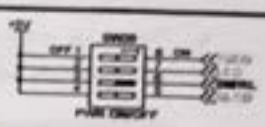
	4x4 Matrix Lines	LPC2148 Lines	4x4 Matrix Keypad
ROW	ROW-0	P1.24	
	ROW-1	P1.25	
	ROW-2	P1.26	
	ROW-4	P1.27	
COLUMN	COLUMN-0	P1.28	
	COLUMN-1	P1.29	
	COLUMN-2	P1.30	
	COLUMN-3	P1.31	

Note: While using Keypad ensure slide switches (SW20-SW27) to **off** position. (The same lines used for both slide switches and matrix keypads)

6.7 – Stepper Motor

The ULN2803A is a high-voltage, high-current Darlington transistor array. The device consists of eight NPN Darlington pairs that feature high-voltage outputs with common-cathode clamp diodes for switching inductive loads. The collector-current rating of each Darlington pair, 500 mA.

ULN2803 is used as a driver for port I/O lines, drivers output connected to stepper motor, connector provided for external power supply if needed.

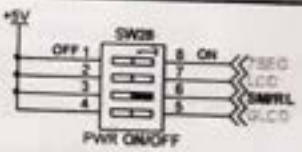
	Stepper Motor(5V)	LPC2148 Lines	Stepper Motor PWR Select
STEPPER MOTOR	COIL-A	P1.16	 JP8 Internal +5V (Stepper Motor)
	COIL-B	P1.17	
	COIL-C	P1.18	
	COIL-D	P1.19	
Make switch SW28 to SM/RL label marking position.			

For Motor/relay section obtain **power** from on-board (internal) or **external supply** through jumper **JP8**.

6.8 – Relay Interface

- ULN2803 is used as a driver for **port** I/O lines, drivers output connected to **relay modules**. Connector provided for external **power supply** if needed.

Relay Module : Port P1 pins (Realy1 – P1.20) and Relay2-P1.21) for relay module, make port pins to high, relay will activated

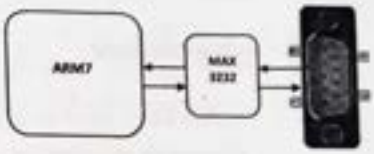
	RELAY SPDT	LPC2148 Lines	RELAY Power Select
RELAY Modules	Relay-1	P1.20	 JP8 Internal +5V (Stepper Motor)
	Relay-2	P1.21	
	Note : Relay selection make switch SW28 to *SM/RL* label marking position		

For Motor/relay section obtain power from on-board (internal) or external supply through jumper JP8.

6.9 - RS-232 Communication

- RS-232 communication enables point-to-point data transfer. It is commonly used in data acquisition applications, for the transfer of data between the microcontroller and a PC.

- The voltage levels of a microcontroller and PC are not directly compatible with those of RS-232, a level transition buffer such as MAX3232 be used.

	UART DB-9 Connector	LPC2148 Processor Lines	Serial Port Section
UART0 (P1) ISP PGM	TXD-0	P0.0	
	RXD-0	P0.1	
UART1 (P2)	TXD-1	P0.8	
	RXD-1	P0.9	

6.10 – Serial EEPROM

The AT24C01A/02/04/08/16 provides 1024/2048/4096/8192/16384 bits of serial electrically erasable and programmable read-only memory (EEPROM) organized as 128/256/512/1024/2048 words of 8 bits each. The device is optimized for use in many industrial and commercial applications where low-power and low-voltage operation are essential.

Features of AT24Cxx:

- Internally Organized 128 x 8 (1K), 256 x 8 (2K), 512 x 8 (4K)
- 2-wire Serial Interface
- Bi-directional Data Transfer Protocol
- 100 kHz (1.8V, 2.5V, 2.7V) and 400 kHz (5V) Compatibility
- Write Protect Pin for Hardware Data Protection
- 8-byte Page (1K, 2K), 16-byte Page (4K, 8K, 16K) Write Modes

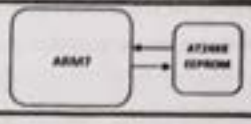
supply through

used in data
a PC.

se of RS-232, a

electrically
1024/2048
mmercial

- Data Retention: 100 Years.

	I2C EEPROM	LPC2148 Lines	Serial EEPROM
AT 24xx	SCL	SCL1 - (P0.11)	
	SDA	SDA1 - (P0.14)	

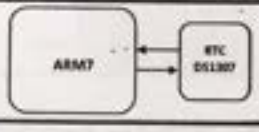
Note : Ensure while using serial EEPROM, GLCD module should be removed from the socket.

6.11 - Real Time Clock (DS1307)

The Real Time Clock (RTC) is a set of counters for measuring time when system power is on, and optionally when it is off. It uses little power in Power-down mode. On the LPC2148, the RTC can be clocked by a separate 32.768 KHz oscillator, or by a programmable prescale divider based on the VPB clock. Also, the RTC is powered by its own power supply pin, VBAT, which can be connected to a battery or to the same 3.3 V supply used by the rest of the device.

Features

- Measures the passage of time to maintain a calendar and clock.
- Ultra Low Power design to support battery powered systems.
- Provides Seconds, Minutes, Hours, Day of Month, Month, Year, Day of Week, Day of Year.
- Dedicated 32 kHz oscillator or programmable pre-scalar from VPB clock.
- Dedicated power supply pin can be connected to a battery or to the main 3.3 V.

	I2C RTC	LPC2148 Lines	Real Time Clock
DS1307	SCL	SCL1 - (P0.11)	
	SDA	SDA1 - (P0.14)	

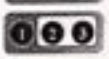
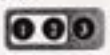
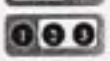
6.12- On-Chip ADC

Basic clocking for the A/D converters is provided by the VPB clock. A programmable divider is included in each converter, to scale this clock to the 4.5 MHz (max) clock needed by the successive approximation process. A fully accurate conversion requires 11 of these clocks.

In **ARM214X Kit**, for testing on-board analog input, port lines P0.29 and P0.30 connected through 10K potentiqmeter selected by jumpers. The signals P0.29 and P0.30 can be used as general purpose pins if the analog inputs are not used and in this case the analog voltages can easily be removed by removing the two jumpers on JP4 and JP5.

Features

- 10 bit successive approximation analog to digital converter (two in LPC2148).
- Input multiplexing among 8 pins.
- Power-down mode | Measurement range 0 to 3 V.
- 10 bit conversion time ≥ 2.44 µs.
- Burst conversion mode for single or multiple inputs.
- Optional conversion on transition on input pin or Timer Match signal.
- Global Start command for both converters (LPC2148 only).

On-Chip ADC	ADC Inputs	LPC2148	ADC Select
POT (R16)	AD0.2	P0.29	JP4  - On-Board ADC1  - External ADC I/P1
POT (R17)	AD0.3	P0.30	JP5  - On-Board ADC2  - External ADC I/P2

6.13- On-Chip Digital-to-Analog Converter (DAC)

DAC Features

- 10 bit digital to analog converter
- Resistor string architecture

- Buffered output
- Power-down mode
- Selectable speed vs. power

DAC Pin Description

Pin	Type	Description
AOUT	Output	**Analog Output.** After the selected settling time after the DACR is written with a new value, the voltage on this pin (with respect to V_{SSA}) is VALUE/1024 * V_{REF}.
V_{REF}	Reference	**Voltage Reference.** This pin provides a voltage reference level for the D/A converter.
V_{DDA}, V_{SSA}	Power	**Analog Power and Ground.** These should be nominally the same voltages as V_3 and V_{SSD}, but should be isolated to minimize noise and error.

Operation

Bits 19:18 of the PINSEL1 register, control whether the DAC is enabled and controlling the state of pin P0.25/AD0.4/AOUT. When these bits are 10, the DAC is powered on and active.

The settling times noted in the description of the BIAS bit are valid for a capacitance load on the AOUT pin not exceeding 100pF. A load impedance value greater than that value will cause settling time longer than the specified time.

ARM2148 Kit

In LPC2148, DAC(P0.25) output terminated at connector JP12.

On-Chip DAC	DAC Output	LPC2148
JP12	Aout	P0.25

6.14 - Temperature Sensor-LM35

The LM35 series are precision integrated-circuit temperature sensors, whose output voltage is linearly proportional to the Celsius (Centigrade) temperature. The LM35 thus has an advantage

over linear temperature sensors calibrated in ° Kelvin, as the user is not required to subtract a large constant voltage from its output to obtain convenient Centigrade scaling.

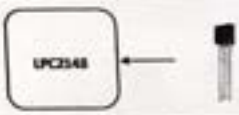
The LM35 does not require any external calibration or trimming to provide typical accuracies of ±¼°C at room temperature and ±¾°C over a full -55 to +150°C temperature range. Low cost is assured by trimming and calibration at the wafer level. It can be used with single power supplies, or with plus and minus supplies. The LM35 is rated to operate over a -55° to +150°C temperature range, while the LM35C is rated for a -40° to +110°C range (-10° with improved accuracy).

Features

- Calibrated directly in ° Celsius (Centigrade)
- Linear + 10.0 mV/°C scale factor
- 0.5°C accuracy guarantee-able (at +25°C)
- Rated for full -55° to +150°C range
- Operates from 4 to 30 volts.

ARM2148 Kit

In LPC2148, LM35 Temp sensor connected at P0.28 (AD0.1)

	Temp Sensor	LPC2148 Lines	Temperature Sensor
LM35	Temp Output	P0.28	

6.15 – Interrupts

The Vectored Interrupt Controller (VIC) takes 32 interrupt request inputs and programmably assigns them into 3 categories, FIQ, vectored IRQ, and non-vectored IRQ. The programmable assignment scheme means that priorities of interrupts from the various peripherals can be dynamically assigned and adjusted.

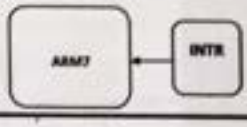
Features

- ARM PrimeCell™ Vectored Interrupt Controller
- 32 interrupt request inputs

- 16 vectored IRQ interrupts
- 16 priority levels dynamically assigned to interrupt requests
- Software interrupt generation

ARM72148 Kit

In LPC2148, two external interrupts lines are terminated at (EXINT1-P0.14) and (EXINT2-P0.15).

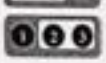
	Interrupts	LPC2148 Lines	Interrupts
Tactile Switch	SW2 – (INT1)	P0.14	
	SW3 – (INT2)	P0.15	

6.16 - Buzzer

A small piezoelectric buzzer on the **ARM214X Kit**, *by* pulling pin P0.7 low, current will flow through the buzzer and a relatively sharp, single-tone frequency will be heard.

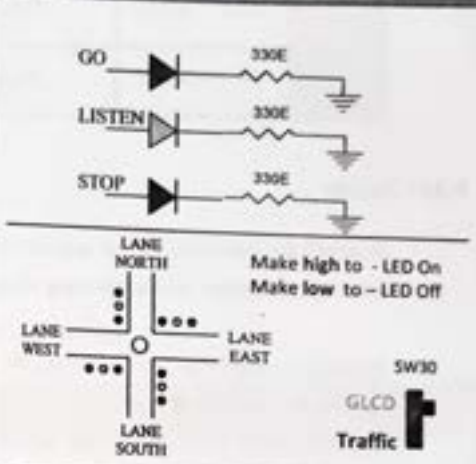
The alternative PWM feature of pin P0.7 (the PWM2 signal) can be used to modulate the buzzer to oscillate around different frequencies. It's not the pulse width feature that is used to change the frequency. Only the volume of the sound will be changed by alternating the pulse width. Instead, it's possible to change the frequency of the PWM signal, and this will also change the frequency of with the buzzer oscillate.

The buzzer can be disconnected by removing jumper JP1, and this is also the default position for this jumper since the buzzer sound can be quite annoying if always left on.

Buzzer	Buzzer	LPC2148	Buzzer Selection
LS1	I/P	P0.7	JP1  - Enable Buzzer  - Disable Buzzer

6.17 – *Traffic Light Controller*

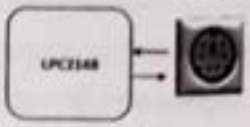
Traffic light controller section consists of 12 Nos. point leds are arranged by 4Lanes. Each lane has Go(Green), Listen(Yellow) and Stop(Red) LED is being placed. Each LED has provided for current limiting resistor to limit the current flows to the LEDs.

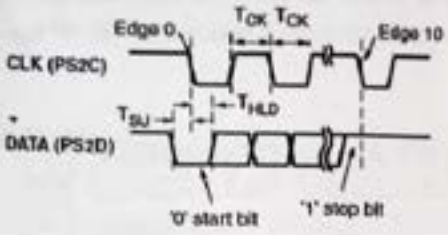
LAN Direction	LPC2148 Lines	LED's	Traffic Light Controller
NORTH	P0.4	D11-Go	
	P0.5	D12-Listen	
	P0.6	D13-Stop	
WEST	P0.7	D14-Go	
	P0.8	D15-Listen	
	P0.9	D16-Stop	
SOUTH	P0.10	D17-Go	
	P0.11	D18-Listen	
	P0.12	D19-Stop	
EAST	P0.13	D20-Go	
	P0.14	D21-Listen	
	P0.15	D22-Stop	

Note :-Make **SW30** to *"Traffic"* label marking position

6.18 - *PS/2 Interface*

The **ARM214X Kit** includes a PS/2 port and the standard 6-pin mini-DIN connector, labeled U11 on the board. User can connect PS/2 Devices like keyboard, mouse to the ARM7 kit. PS/2's DATA (P8) and CLK (P10) lines connected to LPC2148 I/O Lines.

6PIN MINI Connector	PS/2	LPC2148 I/O Lines	PS/2 PORT SELECT
U11 PS/2	DATA	P1.17	
	CLK	P1.16	



PS/2 keyboard connector (MINI-DIN6)

Connector Pin #	Purpose
Pin 1	KBDAT (data)
Pin 2	not used
Pin 3	GND
Pin 4	VCC (+5V)
Pin 5	KBDCLK (clock)
Pin 6	not used

Both a PC mouse and keyboard use the two-wire PS/2 serial bus to communicate with a host device, the ARM7-2148 in this case. The PS/2 bus includes both clock and data. Both a mouse and keyboard drive the bus with identical signal timings and both use 11-bit words that include a start, stop and odd parity bit. However, the data packets are organized differently for a mouse and keyboard. Furthermore, the keyboard interface allows bidirectional data transfers so the host device can illuminate state LEDs on the Keyboard.

The PS/2 bus timing appears as shown in above figure. The clock and data signals are only driven when data transfers occur; otherwise they are held in the idle state at logic High. The timing defines signal requirements for mouse-to-host communications and bidirectional keyboard communications. The attached keyboard or mouse writes a bit on the data line when the clock signal is High, and the host reads the data line when the clock signal is Low.

Keyboard

The keyboard uses open-collector drivers so that either the keyboard or the host can drive the two-wire bus. If the host never sends data to the keyboard, then the host can use simple input pins. A ps/2-style keyboard uses scan codes to communicate key press data nearly all keyboards

in use today are ps/2 style. Each key has a single, unique scan code that is sent whenever the corresponding key is pressed.

The scan codes for most keys appear in below figure. If the key is pressed and held, the keyboard repeatedly sends the scan code every 100 ms or so. When a key is released, the keyboard sends an "f0" key-up code, followed by the scan code of the released key. the keyboard sends the same scan code, regardless if a key has different shift and non-shift characters and regardless whether the shift key is pressed or not. The host determines which character is intended. Some keys, called extended keys, send an "e0" ahead of the scan code and furthermore, they might send more than one scan code. When an extended key is released, an "e0 f0" key-up code is sent, followed by the scan code.

The host can also send commands and data to the keyboard. Below figure provides a short list of some often-used

Commands

Command	Description
ED	Turn on/off Num Lock, Caps Lock, and Scroll Lock LEDs
EE	Echo. Upon receiving an echo command, the keyboard replies with the same scan code "EE".
F3	Set scan code repeat rate. The keyboard acknowledges receipt of an "F3" by returning an "FA", after which the host sends a second byte to set the repeat rate.

FE	Resend. Upon receiving a resend command, the keyboard resends the last scan code sent
FF	Reset. Resets the keyboard

The keyboard sends commands or data to the host only when both the data and clock lines are High, the Idle state, Because the host is the bus master, the keyboard checks whether the host is sending data before driving the bus. The clock line can be used as a clear to send signal. If the host pulls the clock line Low, the keyboard must not send any data until the clock is released. The keyboard sends data to the host in 11-bit words that contain a '0' start bit, followed by eight bits of scan code (LSB first), followed by an odd parity bit and terminated with a '1' stop bi

7. Board Layout

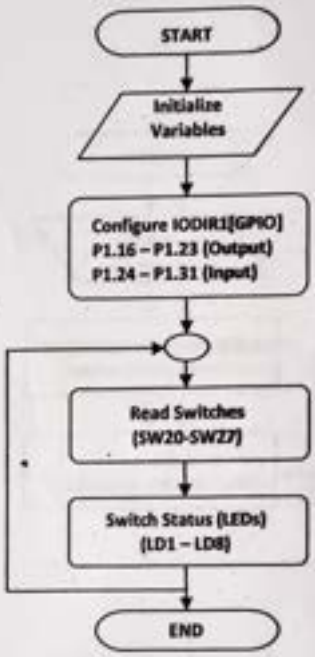
1. 8 Bit LED and Switch Interface
2. Buzzer Relay and Stepper Motor Interface
3. Time delay program using built in Timer / Counter feature
4. External Interrupt
5. 4x4 Matrix Keypad Interface
6. Displaying a message in a 2 line x 16 Characters LCD display
7. ADC and Temperature sensor LM 35 Interface
8. I2C Interface – 7 Segment display
9. I2C Interface – Serial EEPROM
10. Transmission from Kit and reception from PC using Serial Port
11. Generation of PWM Signal

ARM Examples

8-bit Digital Input –Slide Switches

Description	: Program to read switch status and displayed in point LEDs.
Connections	: P1.16 – P1.23 (Point LEDs), Place jumper J4 ('E' Label position) : P1.24 – P1.31 (Slide Switches), Switch SW20 – SW27
Code Path	: PRIMER-ARM7 2148\CODE\LED Switch\OUT\LED_Switch.hex

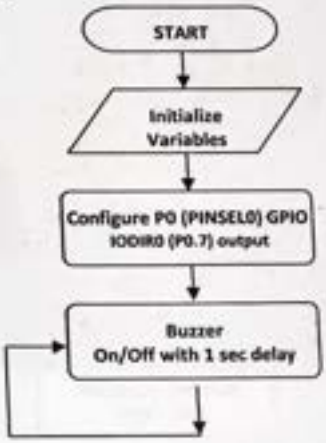
Flow Chart



Buzzer Module Interface

Description	: Program to interface Buzzer.
ARM Pins	: Buzzer (P0.7)
Operation	: ON/Off buzzer with time delay intervals
Code Path	: PRIMER-ARM7 2148\CODE\ Buzzer \OUT\Buzzer.hex
Note	: *Enable* ***Buzzer,*** *Put Jumper* ***JP1*** *to* ***'E'*** *Mode.*

➢ **Flow Chart**



Relay Module Interface

Description : Program to interface Relay.

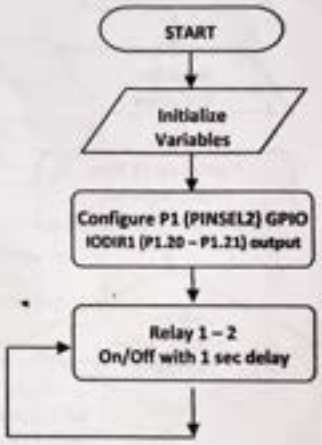
ARM Pins : Relay1 (P1.20) | Relay2 (P1.21)

Operation : Toggle Relays with delay intervals

Code Path : PRIMER-ARM7 2148\CODE\Relay\OUT\Relay.hex

Note : *Enable **Relay**, turn on switch **SW28** at **SM/RL** label mark position*
*Put Jumper **JP8** to **'INT'** Mode to enable Internal Power Supply.*

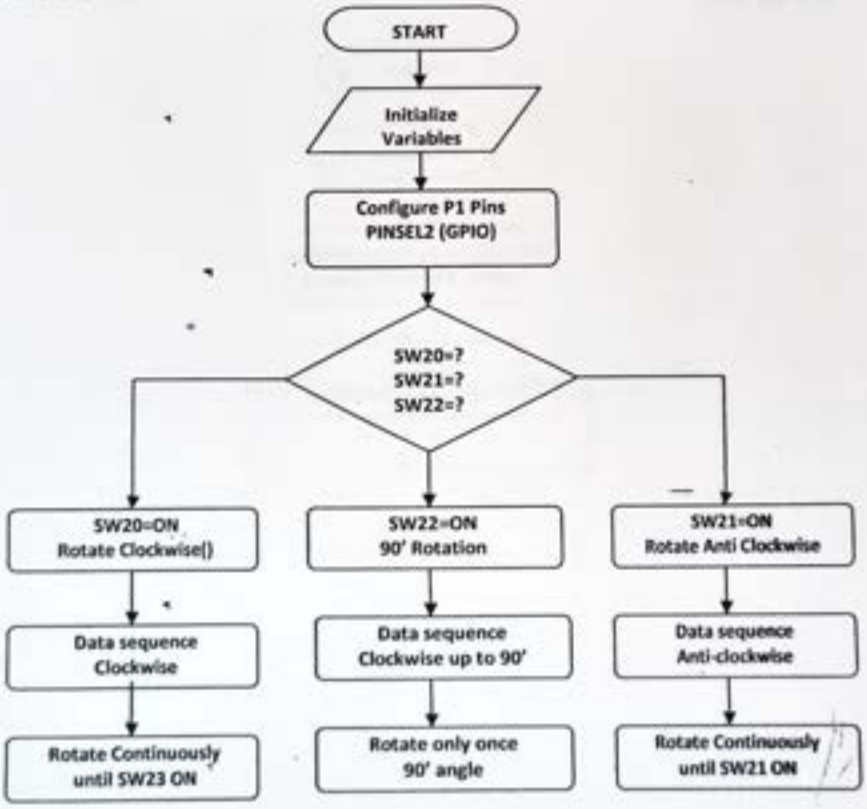
➢ **Flow Chart**



Stepper Motor Interface

Description	: Program to interface Stepper Motor.
ARM Pins	: Motor Coil.A(P1.16) | Coil.B (P1.17) | Coil.C (P1.18) and Coil.D (P1.19)
Operation	: (P1.24)**SW20-Clockwise** | (P1.25)**SW21-Anti-Clockwise** (P1.26)**SW22-90' angle**
Code Path	: PRIMER-ARM7 2148\CODE\Stepper\OUT\Stepper.hex
Note	: *Enable **Stepper Motor**, turn on switch **SW28** at **SM/RL** label mark position Put Jumper **JP8** to **'INT' Mode** to enable Internal Power Supply.*

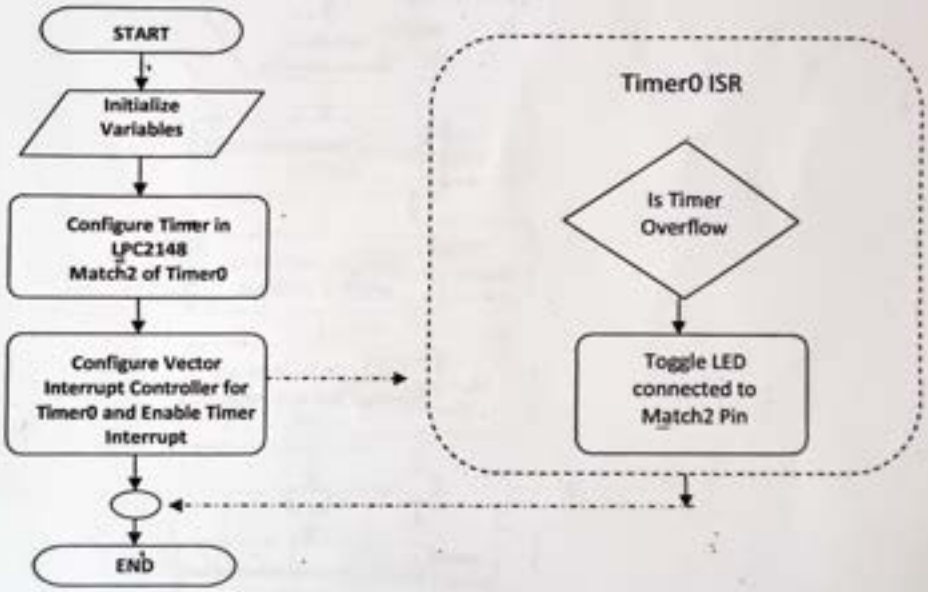
➢ **Flow Chart**



Time Delay Program using Timer / Counter

Description	: Program to toggle LED Based on Timer Delay.
ARM Pins	: On-Board LED's :- P1.16 – P1.23
Code Path	: PRIMER-ARM7 2148\CODE\Timer\OUT\Timer.hex
Note	*: Ensure ARM214xPrimer Kit LED Enable Jumper J4 in 'E' Mode*
	Blinking rate of LED is 1000 ms using Match0 and Match2 of Timer0

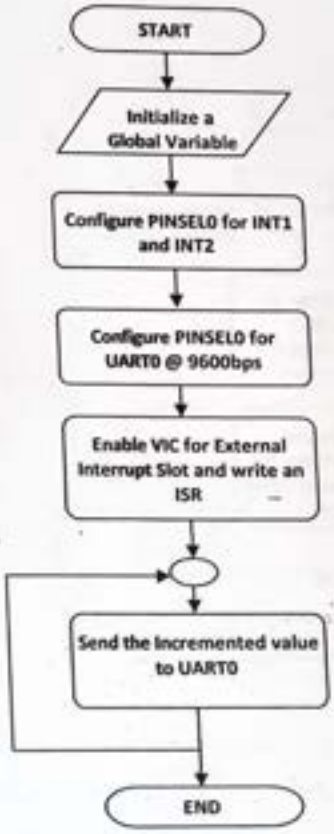
➢ **Flow Chart**



External Interrupt Study

Description	: Program to study external interrupts in LPC2148 MCU
Connections	: **INT 1** (P0.14) –Press to Increment the data **INT 2** (P0.15) – Press to Increment the data Connect Serial Cable at P1 (Board DB9 connector) to PC's DB9 Connector.
Code Path	: PRIMER-ARM7 2148\CODE\Interrupt\OUT\Interrupts.hex
Note	: *Configure Pc's hyper terminal at 9600 baud rate* *** For more information on interrupts kindly refer LPC214x User Manual**

➢ **Flow Chart**



2x16 CHAR LCD Interface

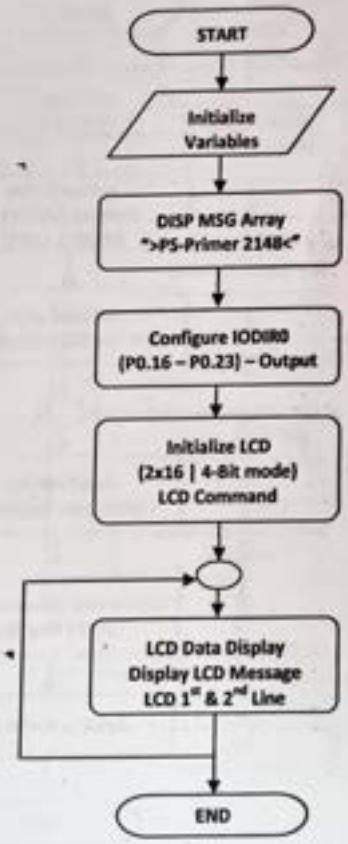
Description : Program to Display Message in 2 lines

ARM Pins : LCD Control - P0.16(RS) | P0.17(RW) | P0.18(E)
LCD Data - P0.19 – P0.22 (D4...D7)

Code Path : PRIMER-ARM7 2148\CODE\LCD 4-bit\OUT\LCD.hex

Note : *Enable **LCD**, turn on switch **SW28** at **LCD** label mark position*

➢ **Flow Chart**



Analog to Digital Conversion (On-Chip ADC) of Temp Sensor

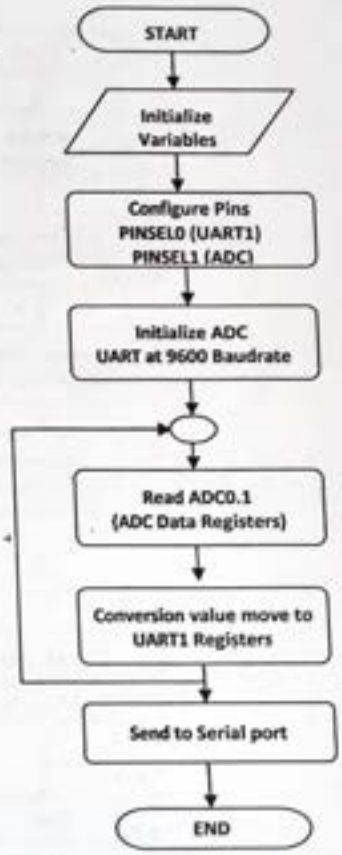
Description : Program to read on-chip ADC value of Temperature sensor LM35 and display in UART1

Connections : **AD0.1** (P0.28) - LM35 Temp Sensor
Connect Serial Cable at **P2** (Board DB9 connector) to PC's DB9 Connector.

Code Path : PRIMER-ARM7 2148\CODE\Temp Sensor\OUT\LM35 Sensor.hex

Note : *Configure Pc's hyper terminal at 9600 baud rate*

➢ **Flow Chart**



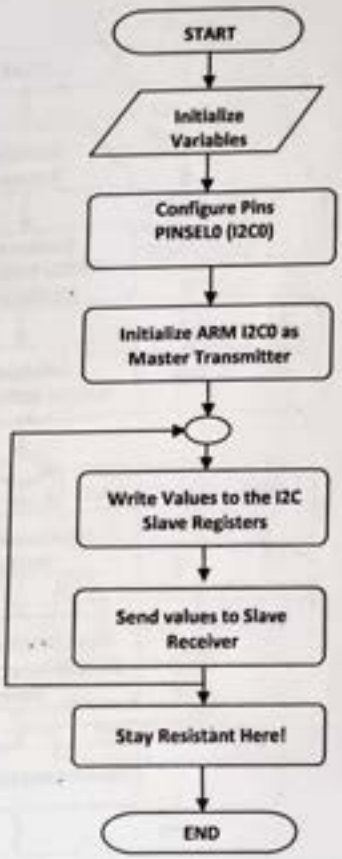
Program for I²C 7-Segment Display Interface

Description : Program to Interface I2C based 7-Segment Display

Pin Details : P0.11 (SCK) | P0.14 (SDA) :: Internal I2C-0 of LPC2148 is Used

Code Path : PRIMER-ARM7 2148\CODE\I2C7 Segment\OUT\I2C 7SEG.hex

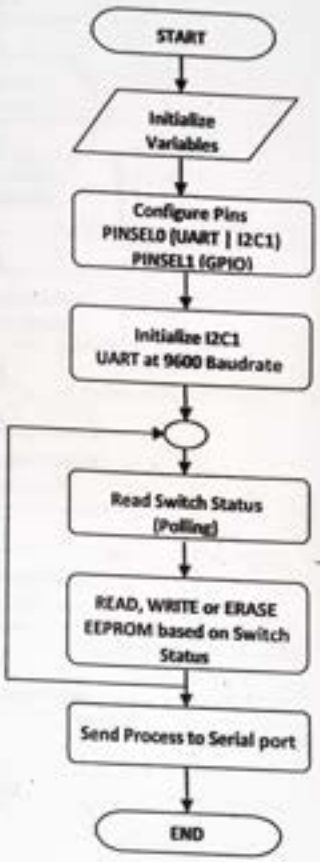
➢ **Flow Chart**



Program for I²C Serial EEPROM Interface

Description	: Program to write some default data and to read the same from a serial EEPROM using ARM7 Internal I2C Bus...
Details	: Connect Serial Cable at P1(Board DB9 connector) to PC's DB9 Connector. **Slide Switch SW20** – Write | **SW21** – Read | **SW22** – Erase
Code Path	: PRIMER-ARM7 2148\CODE\EEPROM\OUT\EEPROM.hex
Note	: *Configure Pc's hyper terminal at 9600 baud rate*

➢ **Flow Chart**



Program to Generate PWM using LPC1248

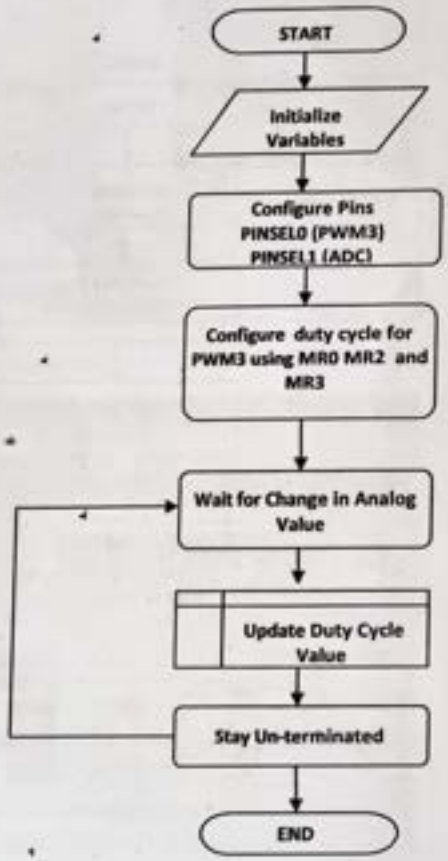
Description : Program to generate PWM using on-chip features

Connections : **AD0.3** (P0.30) – Place jumper JP5 ('I' Label position)
PWM3 is used for Demo (**P0.1**)

Code Path : PRIMER-ARM7 2148\CODE\PWM\OUT\pwm.hex

Note:- To Change the Duty Cycle of the PWM Adjust Trim Pot R17

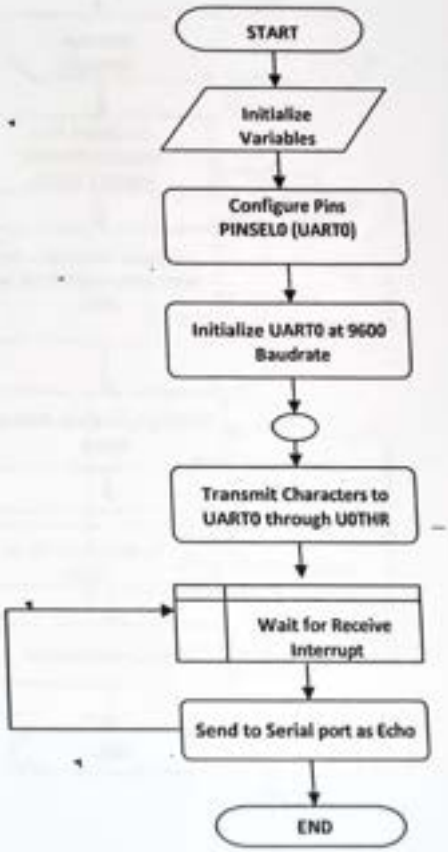
➢ **Flow Chart**



Transmission from Kit and Reception from PC using Serial Port

Description	: Program to put and get characters to and from PC Hyper terminal window at 9600 baud rate.
Connections	: Connect Serial Cable at P1(Board DB9 connector) to PC's DB9 Connector.
Code Path	: PRIMER-ARM7 2148\CODE\UART0\OUT\Uart.hex
Note	: *Configure Pc's hyper terminal at 9600 baud rate*

➢ Flow Chart

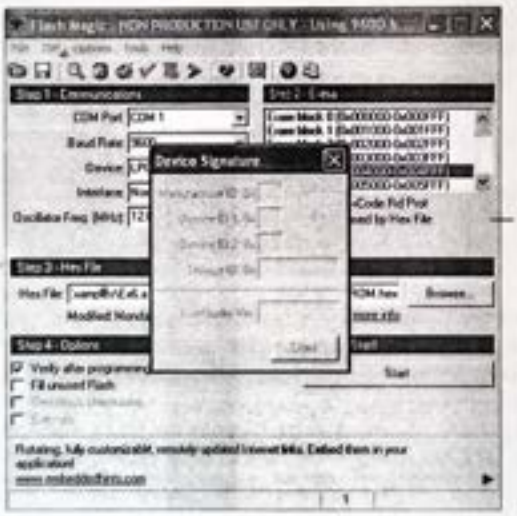


9 - Getting Started with ARM Kit Programming

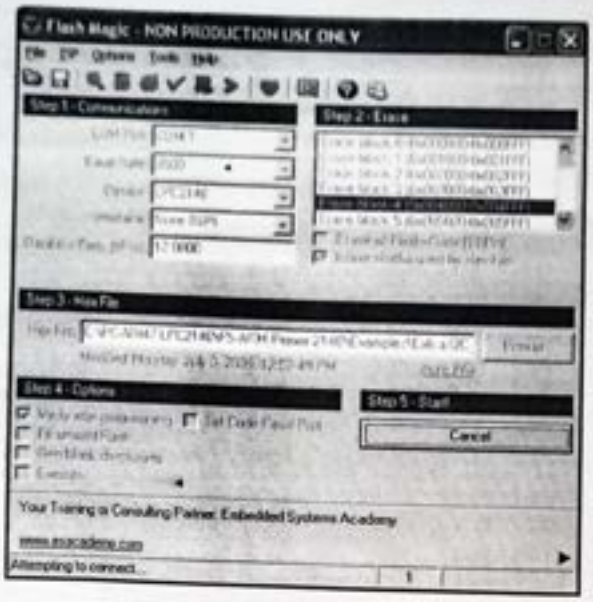
*Note : Ensure slide switch **SW30** near GLCD , in "**GLCD**" label Position **while** in programming mode.*

Step 1: Select Device LPC218

Step 2: Read Device Signature



Step 3: Locate Hex File and Start Programming



10 –Appendix

Appendix A –Product Overview...

Main Board:

- ARM7 Primer 2148 is the main board with most of the on-chip peripherals incorporated on a single slice.
- User Selectable Jumpers
  - ADC0.1 (Temp Sensor) - (P0.28)
  - JP4 – ADC0.2 - (P0.29)
  - JP5 – ADC0.3 – (P0.30)
- J4 – LED Selection (P1.16 – P1.23)
- JP6 – For JTAG operations
- Slide Switch SW1 – For Power Supply Selection (EXT | USB)
- On-Board Interrupt Study | On-Chip RTC Interface | ON-Chip DAC o/p
- 8 Different Slide Switch ⟶ (P1.24 – P1.31)
- Keypad Interface (Pulled Up switches ⟶ P1.24 – P1.31)
- Buzzer Interface ⟶ P0.7
- Relay Interface ⟶ P1.20 | P1.21
- Stepper Motor ⟶ P1.16 – P1.19
- LCD Operates on 4-Bit Mode
  - Control Lines ⟶ (P0.16 – P0.18)
  - Data Lines ⟶ (P0.19 – P0.22)
- GLCD Interface
- I2C Interface (I2C0 and I2C1 Enabled | Devices: RTC | EEPROM | 7-Seg)

Content Details:

- Sample Codes of all Peripherals in \Example\Extra\.. folder
- Experiments solved and provided in \Example\.. folder
- Software (Evaluation | Non-Commercial Products)
- SMPS Adaptor | USB for Power Supply
- JTAG Debugger (Exclusive of the product Content)
- Non-Commercial µC/OS II Kernel Provided
- Datasheets and Reference articles relevant to the product

Trouble Shooting

- It is not advised to connect or disconnect any external devices which are not recommended by the product reference manual.
- It is risky to connect power supply not preferred by the reference user manual
- Disconnecting or Removing ICs on a Powered ON kit is void.
- In case of any malfunction in the product, please let us know.

Pgm (1)

ADDITION

SOURCE CODE:

```
GLOBAL __main
                    AREA main, CODE, READONLY
                    EXPORT __main
                    EXPORT __use_two_region_memory
__use_two_region_memory EQU 0
                    BX     LR
                    ENTRY



;Main loop begins

__main
                    LDR R0,=0X00011111 ;load first input
                    LDR R1,=0X54711212 ;load second input
                    ADD r2,r0,r1

stop        B stop

                    end
```

SUBTRACTION (2)

SOURCE CODE:

```
GLOBAL __main
                    AREA main, CODE, READONLY
                    EXPORT __main
                    EXPORT __use_two_region_memory
```

```
__use_two_region_memory EQU 0
                        BX     LR
                        ENTRY


;Main loop begins

__main
                        LDR R0,=0X00011111 ;load first input
                        LDR R1,=0X54711212 ;load second input
                        SUB r2,r0,r1

stop            B stop

                        end
```

MULTIPLICATION (3)

SOURCE CODE:

```
GLOBAL __main
                        AREA main, CODE, READONLY
                        EXPORT __main
                        EXPORT __use_two_region_memory
__use_two_region_memory EQU 0
                        BX  LR
                        ENTRY


;Main loop begins

__main
```

```
                LDR R0,=0X00011111 ;load first input
                LDR R1,=0X54711212 ;load second input
                MUL  r2,r0,r1

stop        B stop

                end
```

4

ARITHMETIC OPERATION

SOURCE CODE:

```
GLOBAL __main
                AREA main, CODE, READONLY
                EXPORT __main
                EXPORT __use_two_region_memory
__use_two_region_memory EQU 0
                BX     LR
                ENTRY


;Main loop begins

__main
                LDR  R0,=0x00000006 ;load first input
                LDR  R1,=0x00000004 ;load second input
                ADD  r2,R0,R1
                SUBS R3,R0,R1
                MUL  R4,R0,R1

stop        B stop

                end
```

8BIT LED
SOURCE CODE:

```
 GLOBAL __main
                    AREA main, CODE, READONLY
                    EXPORT __main
                    EXPORT __use_two_region_memory
__use_two_region_memory EQU 0
                    BX     LR
                    ENTRY

;Addresses of Registers
IO0DIR              EQU         0xE0028008
IO1DIR              EQU         0xE0028018
IO0PIN              EQU         0xE0028000
IO1PIN              EQU         0xE0028010
IO0SET              EQU         0xE0028004
IO0CLR              EQU         0xE002800C
IO1CLR              EQU         0xE002801C
IO1SET              EQU         0xE0028014
PINSEL0             EQU         0xE002C000
PINSEL1             EQU         0xE002C004
PINSEL2             EQU         0xE002C014
;

__main

                    ldr    r1,=IO1DIR
                    ldr    r2,=0x00FF0000      ;initialize input & output
                    str    r2, [r1]

loop
                    ldr    r1,=IO1PIN
                    LDR    r2, [r1]
                    LSR    R2,#8
                    AND    R2,#0x00ff0000
                    str    r2, [r1]
                    B      loop
                    END
```

BUZZER

SOURCE CODE:

```
GLOBAL __main
                AREA main, CODE, READONLY
                EXPORT __main
                EXPORT __use_two_region_memory
__use_two_region_memory EQU 0
                BX    LR
                ENTRY

;Addresses of Registers
IO0DIR          EQU         0xE0028008
IO1DIR          EQU         0xE0028018
IO0PIN          EQU         0xE0028000
IO1PIN          EQU         0xE0028010
IO0SET          EQU         0xE0028004
IO0CLR          EQU         0xE002800C
IO1CLR          EQU         0xE002801C
IO1SET          EQU         0xE0028014
PINSEL0         EQU         0xE002C000
PINSEL1         EQU         0xE002C004
PINSEL2         EQU         0xE002C014
LEDDELAY  EQU         800000
;

__main
                ldr   r1,=IO0DIR                    ;IO1DIR
OUTPUT CONFIG
                ldr   r2,=0x00FF0000
                str   r2, [r1]


loop
                ldr   r1,=IO0PIN                    ;IO1PIN
                ldr   r2,=0x00FF0000
                str   r2, [r1]
                BL     __delay

                ldr   r1,=IO0PIN
                ldr    r2,=0x00000000
                str   r2, [r1]
```

```
                BL      __delay

                B       loop

;SORF DELAY
;Delay loop
;Constant calculations
;1/30Mhz=1/30us=0.03334us
;1us requires 30 counts ie 30x0.0334=1us
;So 1ms requires 30x1000=30000 counts but the  SUBS and BNE
combination requires 2cy
;So the constant final =30000/2=15000;
__delay

                ldr r0,=0x20000
doloop
                subs r0,#1
                bne doloop
                bx lr
                    END
```

SOURCE CODE:

```
 GLOBAL __main
                AREA main, CODE, READONLY
                EXPORT __main
```

TIMER:

SOURCE CODE:

```
#include <LPC214x.H>
void T0isr(void)      __irq;

void main(void)
{
IODIR1              =       0xFF << 16;  // Configure P1.16 - P1.23 as Output Pins
T0MR0               = 14999999;                 // 1000mSec = 15000.000-1 counts
T0MCR               = 3;                  // Interrupt and Reset on MR0
T0TCR               = 1;                  // Timer0 Enable
VICVectAddr4 = (unsigned)T0isr;                 // Set the timer ISR vector address
VICVectCntl4 = 0x00000024;                          // Set channel
VICIntEnable |= 0x00000010;                         // Enable the interrupt Timer0

/* ------ 1000 ms Delay Used for Toggling LED ------- */

while(1);                                                                   //Stay Here...
}

void T0isr (void)     __irq
{
IOPIN1              =       ~(IOPIN1 & 0x00FF0000);// Toggle LED's for every
1000ms
T0IR          |= 0x00000001;                        // Clear match 0 interrupt
VICVectAddr = 0x00000000;                               // Dummy write to signal
end of interrupt
}
```

SOURCE CODE:

12C 7SERIAL EPROM:

```
#include <lpc214x.h>                          //Includes LPC2148 register
definitions
#define SW3         1<<24
#define SW4         1<<25
#define SW5         1<<26
#define MAX_BUFFER_SIZE 16
#define DEVICE_ADDR 0xA0
#define BLK_0 0x00
#define BLK_1 0x02
#define MAX_BLOCK_SIZE 256
#define LED1_ON() IO1SET=(1<<16)          //I2C write indicator
#define LED2_ON() IO1SET=(1<<17)          //I2C read indicator
#define LED3_ON() IO1SET=(1<<18)          //Comm failure indicator
#define LED4_ON() IO1SET=(1<<19)          //Comm success indicator

#define LED1_OFF() IO1CLR=(1<<16)
#define LED2_OFF() IO1CLR=(1<<17)
#define LED3_OFF() IO1CLR=(1<<18)
#define LED4_OFF() IO1CLR=(1<<19)

#define Fosc        12000000
#define Fcclk       (Fosc * 5)
#define Fcco        (Fcclk * 4)
#define Fpclk       (Fcclk / 4)
```

```
void Delay(unsigned char j);
voidSend_Start(void);
voidSend_Stop(void);
unsigned char Send_I2C(unsigned char *Data,unsigned char Len);
unsigned char Read_I2C(unsigned char *Data,unsigned char Len);
unsigned char Page_Write(unsigned char BLOCK_NUMBER,unsigned char
BLOCK_ADDR,unsigned char *str);
unsigned char Page_Read(unsigned intBLOCK_NUMBER,unsigned char
BLOCK_ADDR);
unsigned char I2C_Status(unsigned char status_code);
unsigned char I2C_WR_Buf[MAX_BUFFER_SIZE]={"ARM7 PRIMER BRD "};
unsigned char I2C_RD_Buf[MAX_BUFFER_SIZE];
unsigned char I2C_ER_Buf[MAX_BUFFER_SIZE]={0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0};
unsigned char Status=0;
unsigned char Status_Flag=0;
void  Delay(unsigned char j)
{
unsignedint  i;
for(;j>0;j--)
 {
for(i=0; i<60000; i++);
 }
}
voidSend_Start()
{
 I2C1CONSET=0x20;
```

```
}
voidSend_Stop()
{
 I2C1CONSET=0x10;
}
// This function sends sequential data to the EEPROM 24LC04
// The buffer size for EEPROM 24LC04 is 16 bytes
// The Len parameter should not exceed this value
unsigned char Send_I2C(unsigned char *Data,unsigned char Len)
{
while(Len)
 {
  I2C1DAT=*Data;
if(I2C_Status(0x28))
  {
return 1;
  }
  Len--;
  Data++;
 }
return 0;
 }
 // This function reads random data from the EEPROM 24LC04
 unsigned char Read_I2C(unsigned char *Data,unsigned char Len)
 {
 while(Len)
```

```
    {
    if(Len==1)  //Last byte
     {
      I2C1CONCLR=0x04;              //set hardware to send nack
    if(I2C_Status(0x58))        //last byte has been received and NACK has been returned
     {
    return 1;
     }
     *Data=I2C1DAT;
    }
   else
    {
    I2C1CONSET=0x04;                    //set hardware to send  ack
   if(I2C_Status(0x50))        //Byte has been received ACK has been returned
    {
   return 1;
    }
    *Data=I2C1DAT;
   }
   Data++;
   Len--;
  }
 return 0;
}
```

```
unsigned char I2C_Status(unsigned char status_code)
{
while(Status_Flag==0);
Status_Flag=0;
if(Status!=status_code)
{
return 1;
}
else
{
return 0;
}
}
unsigned char Page_Write(unsigned char BLOCK_NUMBER,unsigned char
BLOCK_ADDR, unsigned char *str)
{
Send_Start();
if(I2C_Status(0x08))          //Start has been transmitted
{
return 1;
}
I2C1DAT=DEVICE_ADDR | BLOCK_NUMBER;   // Send Address
if(I2C_Status(0x18))                  //Device address, block num and
write has been transmitted
{
return 1;
```

```
 }
 I2C1DAT=BLOCK_ADDR;      // Send block address
 if(I2C_Status(0x28))       //Block address has been transmitted
 {
return 1;
 }
 if(Send_I2C(str,MAX_BUFFER_SIZE))                  //Send Data
 {
Send_Stop();
return 1;
 }
Send_Stop();
return 0;
}
unsigned char Page_Read(unsigned intBLOCK_NUMBER,unsigned char
BLOCK_ADDR)
{
Send_Start();
if(I2C_Status(0x08))       //Start has been transmitted
 {
return 1;
 }
 I2C1DAT=DEVICE_ADDR | BLOCK_NUMBER;    // Send Address
if(I2C_Status(0x18))       //Device address, block num and write has been transmitted
 {
return 1;
```

```
}
I2C1DAT=BLOCK_ADDR;
if(I2C_Status(0x28))        //Block address has been transmitted
{
return 1;
}
Send_Start();              // Repeat Start
if(I2C_Status(0x10))        //Repeated Start has been transmitted
{
return 1;
}
I2C1DAT=DEVICE_ADDR | BLOCK_NUMBER | 0x01;                    //Device
address, block num and read has been transmitted
if(I2C_Status(0x40))        //
{
return 1;
}
if(Read_I2C(I2C_RD_Buf,MAX_BUFFER_SIZE))                    //Receive 16bytes
of Data from EEPROM
{
Send_Stop();
return 1;
}
Send_Stop();
return 0;
}
```

```
void __irq I2C1_Status(void)
{
Status_Flag=0xFF;                    //update status flag
 Status=I2C1STAT;                          //Read Status byte
 I2C1CONCLR=0x28;
VICVectAddr = 0x00;                  //Acknowledge Interrupt
}
void I2C_Init()
{
 PINSEL0|=0x30C00000;
 I2C1CONCLR=0x6C;
 I2C1CONSET=0x40;
I2C1SCLH  =80;
I2C1SCLL  =70;
 /*  Init VIC for I2C1        */
VICIntSelect = 0x00000000;                  // Setting all interrupts as IRQ(Vectored)
 VICVectCntl0 = 0x20 | 19;                  // Assigning Highest Priority Slot to I2C1 and
enabling this slot
 VICVectAddr0 = (unsigned long)I2C1_Status; // Storing vector address of I2C1
VICIntEnable = (1<<19);
}
void UART0_Init(unsigned intbaudrate)
{
unsignedint BAUD_VAL;
 BAUD_VAL = (Fpclk/(16*baudrate));
```

```
 PINSEL0 |= 0x00000005;                          // Enable rx,tx
 U0LCR = 0x00000083;                             // 8 bit data,1 stop
bit,no parity bit
 U0DLL = BAUD_VAL & 0xFF;                        // U0DL for low byte
 U0DLM = (BAUD_VAL>>8);                          // U0DL for high byte
 U0LCR = 0x00000003;                             // DLAB =0
}
void UART0_Txmt(unsigned char Chr)               /* Write character to Serial Port
*/
{
while (!(U0LSR & 0x20));
 U0THR = Chr;
}
unsigned char UART0_Recv(void)                   /* Write character to Serial Port
*/
{
while (!(U0LSR & 0x01));
return (U0RBR);
}
void UART0_puts(unsigned char *string)
{
while(*string)
  UART0_Txmt(*string++);
}
int  main(void)
{
```

```
PINSEL2 = 0;

IO1DIR  = (1<<19) | (1<<18) | (1<<17) | (1<<16);          // Set P1.16, P1.17, P1.18,
P1.19 as Output

IO1DIR &= 0xfC7fffff;

LED1_OFF();LED2_OFF();LED3_OFF();LED4_OFF();

UART0_Init(9600);

I2C_Init();

UART0_puts("\033[2J");

UART0_puts ("********* ARM Primer LPC-2148 I2C EEPROM Demo
**********\n\n\r");

UART0_puts
(">>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>\n\n\r"
);

UART0_puts ("[~] Turn SW 20 ON to Write default data to EEPROM! \n\r");

UART0_puts ("[~] Turn SW 21 ON to Read and Display data from EEPROM! \n\r");

UART0_puts ("[~] Turn SW 22 ON to Erase data from EEPROM \n\r");


while(1)

{

      if ((IOPIN1 & SW3) == 0)                              /*...To Load the Default
Data to the EEPROM ...*/

      {

            LED1_ON();                                              //Write
Indicator

            if(Page_Write(BLK_1,0x00,I2C_WR_Buf))

            {

             UART0_puts("Write Failed");

             LED4_ON();
```

```
        }
        Delay(50);
        LED1_OFF();
        UART0_puts("Data Written Successfully\r\n");
        while (!(IOPIN1 & SW3));
    }
    else if ((IOPIN1 & SW4) == 0)                    /*..To Read the Data
Stored in the EEPROM...*/
        {
            LED2_ON();
    //Read indicator
            if(Page_Read(BLK_1,0x00))
            {
             UART0_puts("Read Failed");
             LED4_ON();
            }
            else
            {
             UART0_puts("Data Read:");
             UART0_puts(I2C_RD_Buf);
             UART0_puts("\r\n");
            }
            Delay(50);
             LED2_OFF();
             while (!(IOPIN1 & SW4));
        }
```

```
else if ((IOPIN1 & SW5) == 0)
                {
                        LED3_ON();
//Write Indicator
                        if(Page_Write(BLK_1,0x00,I2C_ER_Buf))
                        {
                         UART0_puts("Write Failed");
                         LED4_ON();
                        }
                        Delay(50);
                        LED3_OFF();
                        UART0_puts("Data Erase Successfully\r\n");
                        while (!(IOPIN1 & SW5));
                }
```

SOURCE CODE:

I2C 7SEGMENT:

```
#include<LPC214x.h>
#include<stdio.h>
#define MAX        6
#define     AA          2
#define     SI          3
#define     STO         4
#define     STA         5
#define     I2EN  6
//<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<Declarations
>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>
void I2C_ISR(void)__irq;
void Wait (unsignedint);
void I2C_Init (void);
int I2C_Start (void);
int I2C_Write (unsignedchar *Buff, unsignedint Count);
unsignedchar Buff[] =      {0x00,0x27,0x3F/*0*/,0x06/*1*/,0x5B/*2*/,0x4F/*3*/};
unsignedchar index          =      0;


void Delay(void)
{
        unsignedinti,j;
        for(i=0;i<20;i++)
                for(j=0;j<200;j++);
}

void Wait (unsignedint Delay)
{
        while(Delay--);
}

void I2C_Init (void)
{
        I2C0SCLH   =      150; //50%duty cycle .....  I2C Frequency ->100 KHz for
PCLK = 30MHz
        I2C0SCLL   =      150;
        I2C0CONSET        =      1 << I2EN;   //Enable I2C 0

}

int I2C_Start (void)
{
```

```
	I2C0CONCLR	=	1 << STO;
	I2C0CONCLR	=	1 << AA;

	I2C0CONSET	=	1 << STA;

	return 0;
}

int I2C_Write (unsignedchar *Buff, unsignedint Count)
{
	while(Count--)
	{
		I2C0DAT		=	*Buff++;
	}
	return 0;
}
//<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<< Main Function
>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>

void main()
{
	unsignedint i;

	VPBDIV	=	0x02;
	PINSEL0		=	0x00000055; // P0.3 - SDA0 and P0.2	- SCL0

	U0LCR	=	0x83;
	U0DLL	=	97;
	U0DLM	=	0x00;
	U0LCR	=	0x03;

	VICIntSelect = 0<<9;
	VICVectCntl0	= 0x020 | 9 ;
	VICVectAddr0	= (unsignedlong)I2C_ISR;
	VICIntEnable	= 1<<9;

	/*	Before the master transmitter mode
		can be entered, the I2CONSET register must be initialized
	*/

	I2C_Init();

	I2C_Start ();

	for (i=0;i<30;i++)	Wait(10000);
```

```
        I2C0CONCLR      =       1 << SI;

        while(1)
        {
                for (i=0;i<20;i++)   Wait(10000);
        }

}

void I2C_ISR(void) __irq
{
        if (I2C0CONSET & 0x08)
        {

                switch (I2C0STAT)
                {
                        case (0x08)  :      I2C0CONCLR      =       1 << STA;
                                            I2C0DAT         =       0x70;
        //Slave Addr + W
                                            break;

                        case (0x10)  :      I2C0CONCLR      =       1 << STA;
        //Clear START Bit
                                            I2C0DAT         =       0x70;
        //This Status is recieved if "Repeated START" is txd
                                            break;

                        case (0x18)  :      I2C0CONCLR      =       0x20;
        //Clear START Bit
                                            I2C0DAT         =       Buff[index];
                                            index++;

                                            break;

                        case (0x20)  :      I2C0CONCLR      =       0x20;
        //Clear START Bit
                                            I2C0DAT         =       Buff[index];
                                            index++;

                                            break;

                        case (0x28)  :      I2C0CONCLR      =       0x20;
        //Clear START Bit
                                            if (index < MAX)
```

```
                                        {
                                        I2C0DAT         =       Buff[index];
                                                index++;
                                        }
                                        else
                                        {
                                                index = 0;
                                        I2C0CONSET      =       0x10;
        //Send STOP Bit
                                        }
                                        break;

                case (0x30)     :       I2C0CONCLR      =       0x20;
        //Clear START Bit
                                        if (index < MAX)
                                        {
                                        I2C0DAT         =       Buff[index];
                                                index++;
                                        }
                                        else
                                        {
                                                index = 0;
                                                I2C0CONSET      =       0x10;
                //Send STOP Bit
                                                I2C_Start();
                                        }
                                        break;

                case (0x38)     :       I2C0CONSET      =       0x20;
                                        break;
        }
    }
    I2C0CONCLR      =       1 << SI;
    VICVectAddr     =       0x00;
}
```

SOURCE CODE:

I2C SEGMENT COUNTER:

```
#include<LPC214x.h>
#include<stdio.h>

#define MAX        6
//<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<Declarations
>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>
void I2C_ISR(void)__irq;
void Wait (unsignedint);
void I2C_Init (void);
int I2C_Start (unsignedintSlave_Addr);
int I2C_Write (unsignedchar *Buff, unsignedint Count);


unsignedchar Buff[] =       {0x00,0x27,0x3F,0x3F,0x3F,0x3F};
unsignedchar Segment[]   =
        {0x3F/*0*/,0x06/*1*/,0x5B/*2*/,0x4F/*3*/,0x66/*4*/,

0x6D/*5*/,0x7D/*6*/,0x07/*7*/,0x7F/*8*/,0x6F/*9*/};
//unsigned char Buff[]      =      {0x00,0x27,0x5B,0x4F,0x06,0x3F};
unsignedchar index          =      0;
unsignedint flag=0;

unsignedchar all_chars[] = "0123456789-.ACEFGHIJLPUYbcdfhinortu°";

char digit[] = {

        0x3F,0x06,0x5B,0x4F,0x66,0x6D,0x7D,0x07,0x7F,0x6F,

        0x77,0x7C,0x58,0x39,0x5E,0x79,0x71,0x3D,0x76,0x0E,
                                  0x38,0x54,0x5C,0x73,0x50,0x78,0x3E,0x1C,0x6E
                        };

#define     AA       2
#define     SI       3
#define     STO      4
#define     STA      5
#define     I2EN  6


void Delay(void)
{
        unsignedinti,j;

        for(i=0;i<150;i++)
```

```
        for(j=0;j<900;j++);
}

void Wait (unsignedint Delay)
{
        while(Delay--);
}

void I2C_Init (void)
{
        I2C0SCLH    =    150;        //50%duty cycle .....  I2C Frequency ->100
KHz for PCLK = 30MHz
        I2C0SCLL    =    150;
        I2C0CONSET      =    1 << I2EN;   //Enable I2C 0

}

int I2C_Start (unsignedintSlave_Addr)
{
        I2C0CONCLR    =    1 << STO;
        I2C0CONCLR    =    1 << AA;

        I2C0CONSET    =    1 << STA;
        //I2C0CONSET    =    1 << SI;


        //while(I2C0CONSET & 0x08); //wait Till SI is cleared
        return 0;
}

int I2C_Write (unsignedchar *Buff, unsignedint Count)
{
        while(Count--)
        {
                I2C0DAT        =    *Buff++;
        }
        return 0;
}
//<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<  Main Function
>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>


void main()
{
        unsignedinti,j,k,l;
```

```
VPBDIV      =     0x02;
PINSEL0     =     0x00000055;// P0.3 - SDA0 and P0.2     - SCL0

U0LCR       =     0x83;
U0DLL       =     97;
U0DLM       =     0x00;
U0LCR       =     0x03;

VICIntSelect = 0<<9;
VICVectCntl0       = 0x020 | 9 ;
VICVectAddr0       = (unsignedlong)I2C_ISR;
VICIntEnable       = 1<<9;

/*    Before the master transmitter mode
      can be entered, the I2CONSET register must be initialized
*/

//    I2C0CONSET      =     0x40;          //Master Only Mode
//    I2C0CONSET      =     0x20;          //Start I2C comm -> STA = 1

I2C_Init();

I2C_Start (0x70);

for (i=0;i<30;i++)   Wait(10000);

I2C0CONCLR      =     1 << SI;
i=0;
j=0;
k=0;
l=0;
//    U0THR       =     'A';
while(1)
{
      Delay();
      Delay();
      flag++;
      if (flag > 10)
      {
            flag = 0;
            i++;
            if (i>9)
            {
                  i=0;
                  j++;
                  if (j>9)
```

```
                        {
                            j=0;
                            k++;
                            if (k>9)
                            {
                                k=0;
                                l++;
                                if (l>9)
                                    i=j=k=l=0;
                            }
                        }
                    }
                }
                Buff [2] = Segment[i];
                Buff [3] = Segment[j];
                Buff [4] = Segment[k];
                Buff [5] = Segment[l];
        }

}

void I2C_ISR(void) __irq
{
        if (I2C0CONSET & 0x08)
        {

                switch (I2C0STAT)
                {
                        case (0x08)  :   I2C0CONCLR     =   1 << STA;
                                         I2C0DAT        =   0x70;
        //Slave Addr + W
                                         U0THR          =   I2C0STAT;
                                         //I2C0CONCLR   =   0x08;
        //Clear SI
                                         break;

                        case (0x10)  :   I2C0CONCLR     =   1 << STA;
        //Clear START Bit
                                         I2C0DAT        =   0x70;
        //This Status is recieved if "Repeated START" is txd
                                         U0THR          =   'C';
                                         break;

                        case (0x18)  :   I2C0CONCLR     =   0x20;
        //Clear START Bit
```

```
                                    I2C0DAT   =   Buff[index];
                                    U0THR     =   'D';
                                    index++;

                                    break;

            case (0x20)  :    I2C0CONCLR    =   0x20;
//Clear START Bit
                                    I2C0DAT   =   Buff[index];
                                    U0THR     =   'E';
                                    index++;

                                    break;

            case (0x28)  :    I2C0CONCLR    =   0x20;
//Clear START Bit
                                    if (index < MAX)
                                    {
                                        I2C0DAT=  Buff[index];
                                        U0THR     =   'F';
                                        index++;
                                    }
                                    else
                                    {
                                        index = 0;
                                        I2C0CONSET      =0x10;
        //Send STOP Bit
                                        U0THR     =   'G';
                                        I2C_Start(0x70);
                                        I2C0CONCLR      =   1 <<
SI;
                                    }
                                    break;

            case (0x30)  :    I2C0CONCLR    =   0x20;
//Clear START Bit
                                    if (index < MAX)
                                    {
                                        I2C0DAT=  Buff[index];
                                        U0THR     =   'H';
                                        index++;
                                    }
                                    else
                                    {
                                        index = 0;
```

```
                                        I2C0CONSET      =0x10;
            //Send STOP Bit
                                        U0THR       =   'I';
                                        I2C_Start(0x70);
                                    }
                                    break;

                case (0x38)  :      I2C0CONSET      =   0x20;
                                    break;
            }
        }
        I2C0CONCLR      =   1 << SI;
        VICVectAddr     =   0x00;
}
```

SOURCE CODE:
LCD:

```
#include<LPC214x.H>

#define RS    0x10000
#define RW    0x20000
#define EN    0x40000

voidlcd_cmd  (unsignedchar);
voidlcd_data (unsignedchar);
voidlcd_initialize (void);
voidlcd_display (void);
void LCD4_Convert(unsignedchar);

constunsignedcharcmd[4] = {0x28,0x0c,0x06,0x01};   //lcd commands

unsignedcharmsg[] = {">PS-Primer 2148<"};                    //msg
unsignedchar msg1[]= {":: LCD Demo! ::"};
    //msg1


void delay(unsignedint n)
{
```

```
    unsignedinti,j;
    for(i=0;i<n;i++)
    {
        for(j=0;j<12000;j++)
        {;}
    }
}
//-------------------------------
//          LCD Command Send
//-------------------------------
voidlcd_cmd(unsignedchar data)
{
    IOCLR0    |= RS;              //0x1000;          //RS
    IOCLR0    |= RW;                    //0x2000;          //RW
    LCD4_Convert(data);
}

voidlcd_initialize(void)
{
    int i;
    for(i=0;i<4;i++)
    {
        IOCLR0 = 0xF << 19;                    //IOCLR 0/1
        lcd_cmd(cmd[i]);
        delay(15);
    }
}

//-------------------------------
//          LCD Data Send
//-------------------------------
voidlcd_data (unsignedchar data)
{
    IOSET0    |= RS;              //0x1000;          //RS
    IOCLR0    |= RW;                    //0x2000;          //RW
    LCD4_Convert(data);
}

//-------------------------------
//          LCD Display Msg
//-------------------------------
voidlcd_display (void)
```

```
{
        char i;

        /* first line message */
        lcd_cmd(0x80);
        delay(15);
        i=0;
        while(msg[i]!='\0')
        {
                delay(5);
                lcd_data(msg[i]);
                i++;
        }
        delay(15);

        /* second line message */

        lcd_cmd(0xc0);
        delay(15);
        i=0;
        while(msg1[i]!='\0')
        {
                delay(5);
                lcd_data(msg1[i]);
                i++;
        }
        delay(15);
}

void LCD4_Convert(unsignedchar c)
{

        if(c & 0x80) IOSET0 = 1 << 22; else IOCLR0 = 1 << 22;
        if(c & 0x40) IOSET0 = 1 << 21; else IOCLR0 = 1 << 21;
        if(c & 0x20) IOSET0 = 1 << 20; else IOCLR0 = 1 << 20;
        if(c & 0x10) IOSET0 = 1 << 19; else IOCLR0 = 1 << 19;

        IOSET0      = EN;
                //0x4000;               //EN
        delay(15);
        IOCLR0      = EN;
                //0x4000;               //EN

        if(c & 0x08) IOSET0 = 1 << 22; else IOCLR0 = 1 << 22;
        if(c & 0x04) IOSET0 = 1 << 21; else IOCLR0 = 1 << 21;
        if(c & 0x02) IOSET0 = 1 << 20; else IOCLR0 = 1 << 20;
```

```
        if(c & 0x01) IOSET0 = 1 << 19; else IOCLR0 = 1 << 19;

        IOSET0      = EN;
                //0x4000;           //EN
        delay(15);
        IOCLR0      = EN;
                //0x4000;           //EN
}

void main()
{
        PINSEL1     =     0;
        //Configure P0.16 - P0.31 as GPIO
        IODIR0      =     0xFF << 16;  //Configure Pins P0.16 - P0.22 as Output Pins
        lcd_initialize();                          //Initialize LCD!
        lcd_display();                             //Display Message in LCD
        while(1);
}
```

SOURCE CODE:
LED:

```
#include<LPC214x.h>
#include<stdio.h>

#define LED 16
#define Switch 24
/*************************************     Declarations
*************************************/
void Delay(int);
/*****************************************************************************
***************************/

int main(void)
{
        unsignedchar Status=1;
```

```
        PINSEL2     &=    0xFFFFFFF3;       //Configure P1.16 - P1.31 as GPIO

        IO1DIR      =     0x00 << Switch;   //Configure P1.24 - P1.31 as Input
        IO1DIR      |=    0xFF << LED;      //Configure P1.16 - P1.23 as Output

        IO1PIN      =     0;

        while(1)
        {
              Status = 1;
              IOSET1      =     0xFF << LED;
              Delay (10);Delay (10);
              IOCLR1      =     0xFF << LED;
              Delay (10);Delay (10);Delay (10);

              while (~Status)
              {
                    Status =    ((IO1PIN & (0xFF << Switch)) >> Switch);
                    IO1PIN      =     ((~Status) << LED);
              }

        }
}

void Delay(int n)
{
        intp,q;

        for(p=0;p<n;p++)
        {
              for(q=0;q<0x9990;q++);
        }
}
```

INTERRUPTS:

SOURCE CODE:

```
#include<lpc214x.h>
#include<stdio.h>
intvolatile EINT1    =      0;
intvolatile EINT2    =      0;
void ExtInt_Serve1(void)__irq;
void ExtInt_Serve2(void)__irq;

/*-----------------------------------< INT2 Initialization >-----------------------------*/
void ExtInt_Init2(void)
{
        EXTMODE |= 4;
        //Edge sensitive mode on EINT2
        EXTPOLAR = 0;
        //Falling Edge Sensitive
        PINSEL0 |= 0x80000000;
        //Enable EINT2 on P0.15
        VICVectCntl1 = 0x20 | 16;                                    //Use VIC1
for EINT2 ; 16 is index of EINT2
        VICVectAddr1 = (unsignedlong) ExtInt_Serve2;                 //Set Interrupt
VecAddr in VIC1
        VICIntEnable |= 1<<16;
        //Enable EINT2
}

/*-----------------------------------< INT1 Initialization >-----------------------------*/
void ExtInt_Init1(void)
{
        EXTMODE |= 2;
        //Edge sensitive mode on EINT1
        EXTPOLAR = 0;
        //Falling Edge Sensitive
        PINSEL0 |= 0x20000000;
        //Enable EINT2 on P0.14
        VICVectCntl0 = 0x20 | 15; //Use VIC0 for EINT2 ; 15 is index of EINT1
VICVectAddr0 = (unsignedlong) ExtInt_Serve1;        //Set Interrupt VecAddr in VIC0
        VICIntEnable |= 1<<15;
                //Enable EINT1
}

/*-----------------------------------< Serial Initialization >-----------------------------*/
voidSerial_Init(void)
{
        PINSEL0 |=  0X00000005;                          //Enable Txd0 and Rxd0
        U0LCR       =       0x00000083; //8-bit data, no parity, 1-stop bit
```

```
        U0DLL       =       0x00000061; //XTAL = 12MHz (pclk = 60 MHz, VPB =
15MHz(pclk=cclk/4)
        U0LCR       =       0x00000003;         //DLAB = 0;
}
/*-----------------------------------<  Delay Function >-----------------------------------*/
voidDelayMs(unsignedint count)  {

unsignedinti,j;
for(i=0;i<count;i++)
        {
                for(j=0;j<1000;j++);
        }
}

/*-----------------------------------<  Main Function >-----------------------------------*/

void main(void)
{
        Serial_Init();
        ExtInt_Init1();                                         //Initialize Interrupt 1
        ExtInt_Init2();                                         //Initialize Interrupt 2

        putchar(0x0C);
        printf ("****************** External Interrupt Demo
*****************************\n\n\r");
        DelayMs(100);
        printf (">>> Press Interrupt Keys SW2(INT1) and SW3(INT2) for Output...
\n\n\n\r");
        DelayMs(100);

        while(1)
        {
                DelayMs(500);
                printf("INT1 Generated : %d  |  INT2 Generated : %d \r", EINT1, EINT2);
                DelayMs(500);
        }

}

/*-----------------------------------<  Interrupt Sub-Routine >-----------------------------------*/

void ExtInt_Serve1(void)__irq
{
        ++EINT1;
        EXTINT |= 2;
        VICVectAddr = 1;
```

```
}

void ExtInt_Serve2(void)__irq
{
        ++EINT2;
        EXTINT |= 4;
        VICVectAddr = 0;
}
```